DEVELOPMENT PLAN · 4 WEEKS · 2 TO 27 NOVEMBER 2026

Naive GraphRAG Delivery Plan

In four weeks ASE designs, generates, indexes and evaluates a working naive GraphRAG agent over a real document corpus in FalkorDB. Both the indexer and the query agent are generated from library patterns, not hand-written.

Assumptions: weeks start Monday 2 November 2026; owners are the architecture owner (A) and the delivery owner (D), with points sized for two to three engineers. Ticket ids (GR-n) are local; JIRA assigns its own keys on import.

Goal and definition of done

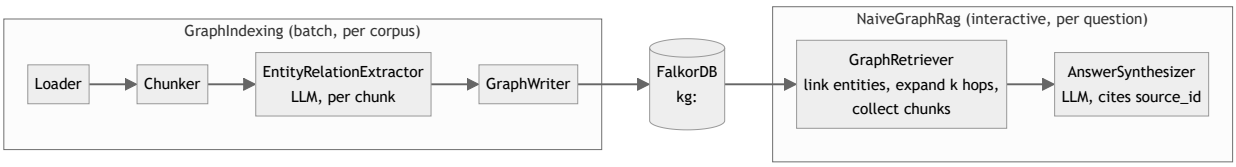
"Naive" means the simplest GraphRAG that is still honest: entities and relations are extracted from chunks at indexing time; at query time, question terms are linked to graph entities, their neighbourhood is expanded a bounded number of hops, and the chunks attached to those entities are passed to the synthesizer with citations. No community summaries, no global search, no graph learning.

The 4-week goal is met when all of these hold:

1. `cli run --project RS-11 --online` selects `NaiveGraphRag`, passes the gate and the sweep, generates the indexer and the agent, builds the `kg:RS-11` graph in FalkorDB, and answers the RS-11 eval set with citations.
2. On the RS-11 eval set: cross-document recall and citation recall beat the word-overlap baseline on the same questions; unanswerable questions are refused.
3. The same run works offline (no key, no FalkorDB) with a deterministic extractor and an in-memory graph, so CI stays green.
4. `cli selftest` is OK; regenerating the programs gives byte-identical files; `models/` changed only through the reviewed library commit in week 1.
5. Every design choice below is a Decision node with owner, date, reason and requirement.

Target design

Two new library patterns. `GraphIndexing` runs once per corpus (batch workload); `NaiveGraphRag` runs per question (interactive). Keeping them separate lets the analysis budget each on its own terms.



Graph schema (`kg:<project>`)

ELEMENT	PROPERTIES	PURPOSE
`(:Document)`	id, title, path	Source file
`(:Chunk)`	id (`<doc>#<n>`), text, section, page	The unit that gets cited
`(:Entity)`	key (normalised), name, type, aliases	Party, defined term, clause, product, …
`(:Document)-[:HAS_CHUNK]→ (:Chunk)`	order	Provenance
`(:Chunk)-[:MENTIONS]→ (:Entity)`		Entity to evidence
`(:Entity)-[:RELATED]→ (:Entity)`	type, chunk_id	Relation, with the chunk that states it

Indexes: full-text on `Entity.name` and `Entity.aliases` ; range on `Chunk.id` and `Entity.key` . All Cypher is parameterised. The `kg:*` graphs are product data, kept apart from ASE's own `ase` and `ase-designs` graphs.

Expected analysis numbers (declared estimates)

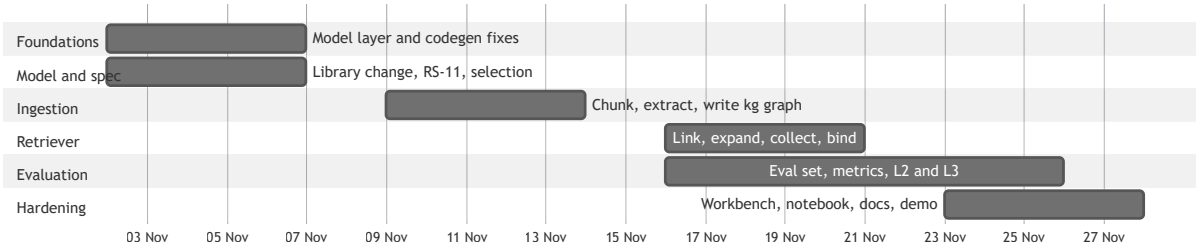
PATTERN	STAGES	LLM CALLS	LATENCY MS	TOKENS
NaiveRag (reference)	DenseRetriever 120 + AnswerSynthesizer 1,400	1	1,520	2,200
NaiveGraphRag	GraphRetriever 250 + AnswerSynthesizer 1,400	1	1,650	2,200

GraphRetriever's 250 ms is the library's existing estimate. Indexing cost scales with corpus size (one extraction call per chunk) and is reported by the ingestion report, not the per-question budget.

Design decisions to record in week 1

#	DECISION	PROPOSED CHOICE	OWNER
DG-1	Bring GraphRetriever forward from I5 (where the roadmap and RS-07 place it)	Yes, single-tenant only; tenancy, fusion with dense and sparse, and RS-07 stay in I5	A
DG-2	Model indexing as its own pattern	`GraphIndexing` pattern, so its cost and provenance are analysed and its program is generated	A
DG-3	Entity linking at query time	FalkorDB full-text index over names and aliases; no query-time LLM call. Embedding-based linking is a later option	A + D
DG-4	Expansion depth	`maxHops = 2`, node cap 200, as stage-instance attributes on the design	D
DG-5	Graph naming and ownership	`kg:<project-id>`, added to the launcher's owned graphs	D
DG-6	Offline path	Deterministic rule-based extractor plus an in-memory graph retriever; same interface as the live path	D
DG-7	Spec id	RS-11. PR #1 used RS-11 to RS-15 for its unmerged examples; those get renumbered if they are ever adopted	A

Timeline



WEEK	FOCUS	STORIES	POINTS	A / D POINTS
Week 1 (2 to 6 Nov)	Foundations, library, spec	12	34	10 / 24
Week 2 (9 to 13 Nov)	Ingestion and graph build	8	25	0 / 25
Week 3 (16 to 20 Nov)	Retriever, eval set, metrics	8	24	3 / 21
Week 4 (23 to 27 Nov)	End to end, inspection, review	9	23	5 / 18
Total		37	106	

Week 1 · 2 to 6 November

12 stories · 34 points

Make every later step possible: provider-agnostic model calls, generated programs that can carry new state and parallel branches, and the library, spec and selection for GraphRAG.

GR-E1 · FOUNDATIONS AND CODEGEN FIXES

GR-1 Provider-agnostic model factory

3 pts · Delivery owner

Add sysmlagents/llm.py with lm(role) returning dspy.LM from SYSMLAGENTS_MODEL (role=task) or SYSMLAGENTS_REFLECTION_MODEL (role=reflection). Offline returns the DummyLM. runtime_stub.configure_lm and codegen's configure() use it; the offline/online choice no longer depends on OPENAI_API_KEY.

ACCEPTANCE CRITERIA

- Switching provider is a change of one environment variable
- All tests pass with no API key set
- No provider SDK imported in pipeline, agent or codegen modules (grep check in a test)

GR-2 Codegen: fan-in reducers so parallel retrievers can join

5 pts · Delivery owner

Generated nodes return state deltas instead of the whole state; chunks from parallel branches merge through an Annotated reducer keyed by source_id; fusion waits for all inputs. Port and review the fan-in fix from PR #1. Regenerate generated/.

ACCEPTANCE CRITERIA

- RerankHybridRag and AgenticRag programs run offline without InvalidUpdateError
- Regenerating twice gives byte-identical files
- Notebooks re-run green

GR-3 Codegen: declare every stage output in the state schema

3 pts · Delivery owner

Build RagState from the union of STAGE_CONTRACTS outputs plus retrieval keys, so new keys such as verdict, entities and subgraph are not silently dropped by LangGraph.

ACCEPTANCE CRITERIA

- CorrectiveRag skips the web fallback when the relevance verdict is 'sufficient' (test)
- A new contract output appears in RagState without hand edits

Depends on: GR-2

GR-4 Codegen: persist the loop counter in a node

2 pts · Delivery owner · hygiene

Move the iteration increment out of the routing function into a node so maxIterations is enforced. Hygiene item; does not block GraphRAG.

ACCEPTANCE CRITERIA

- An always-ungrounded SelfRag stops after 3 iterations (test)

Depends on: GR-2

GR-5 Declare runtime dependencies

1 pts · Delivery owner

Add pypdf and sentence-transformers (already installed) to ASE requirements; remove the stale llm.py comment.

ACCEPTANCE CRITERIA

- Fresh environment installs cleanly; pip check reports no problems

GR-E2 · LIBRARY, SPEC AND SELECTION

GR-6 Record GraphRAG design decisions DG-1 to DG-7

2 pts · Architecture owner

Write the decisions in this plan as Decision records (owner, date, reason, requirement) and add a dated note to ROADMAP.md that a single-tenant GraphRetriever moves forward from I5.

ACCEPTANCE CRITERIA

- Seven Decision records exist and are linked to RS-11 requirements
- ROADMAP.md notes the change and its reason

GR-7 Library change: GraphIndexing and NaiveGraphRag patterns

3 pts | Architecture owner

In rag-patterns.sysml add items Entity and Relation; stage defs Loader, Chunker, EntityRelationExtractor (llmCalls 1, declared latency and tokens per chunk), GraphWriter; attributes maxHops and nodeCap on GraphRetriever; patterns GraphIndexing (loader to chunker to extractor to writer) and NaiveGraphRag (graph to synth) with satisfy allocations. One commit labelled as a library change.

ACCEPTANCE CRITERIA

- Structural gate passes on the library
- Analysis computes NaiveGraphRag at 1 call, 1,650 ms, 2,200 tokens, bounded, provenance preserved
- Sweep has no errors; specs/README numbers table regenerated
- cli selftest OK

Depends on: GR-6

GR-8 Codegen: contracts and programs for the new stage kinds

5 pts | Delivery owner

Add a STAGE_CONTRACT for EntityRelationExtractor (chunk text to entities and relations as JSON) and RETRIEVAL_STUBS docs for Loader, Chunker, GraphWriter. generate_all emits graph_indexing.py and naive_graph_rag.py.

ACCEPTANCE CRITERIA

- Both programs import and run offline with stub implementations
- Byte-identical regeneration
- No TODO signatures in either program

Depends on: GR-3, GR-7

GR-9 Spec RS-11 Contract Relationship Assistant (single tenant)

3 pts | Architecture owner

Write docs/specs/RS-11 in the RS format: submitted section with FR/QR/C ids (relate clauses via parties, defined terms and cross-references; cite document, clause, page; refuse when unsupported; 3 s latency; 5,000 tokens) and a hidden oracle naming NaiveGraphRag, expected numbers, questions and metrics.

ACCEPTANCE CRITERIA

- read_spec parses it; budgets derive to 3000 ms and 5000 tokens
- Oracle lists expected pattern, rejections and metrics

Depends on: GR-6

GR-10 RS-11 fixture corpus with gold triples

3 pts · Delivery owner

docs/specs/data/RS-11/corpus: 20 to 30 short synthetic contracts and amendments with shared parties, defined terms and cross-references, plus gold/triples.json and gold/answers.json. If an LLM helps write it, record the model, prompt and seed.

ACCEPTANCE CRITERIA

- Workspace.create copies the corpus into inputs/
- Gold triples cover every multi-hop eval question
- Architecture owner reviewed and approved the corpus

Depends on: GR-9

GR-11 RuleSelector: rules and card for NaiveGraphRag

2 pts · Delivery owner

Add RULES and PATTERN_CARDS entries (relate, cross-reference, parties, defined terms, across documents, relationships, multi-hop).

ACCEPTANCE CRITERIA

- RS-11 selects NaiveGraphRag with a rationale and rejections
- RS-01 still selects NaiveRag; existing tests unchanged

Depends on: GR-7, GR-9

GR-12 Evolution model: index step with @Delta and @Realises

2 pts · Architecture owner

Add an index step (realises R1) between codegen and eval in the current workbench entry of ase-evolution.sysml, with its @Delta; mirror the tool in ase-system.sysml.

ACCEPTANCE CRITERIA

- cli selftest OK, including the orchestrator edge check
- Generated orchestrator includes the index step

Depends on: GR-7

Week 2 · 9 to 13 November

8 stories · 25 points

Turn a corpus into a knowledge graph in FalkorDB, through a generated indexing program, with an offline twin for tests.

GR-E3 · CORPUS INGESTION AND GRAPH BUILD

GR-13 Loaders for txt, md and pdf

2 pts Delivery owner

rag_runtime/loaders.py returns documents with id, title, path and page-aware text (pypdf for PDF).

ACCEPTANCE CRITERIA

- Unit tests over one file of each type
- Page numbers preserved for PDF

Depends on: GR-5

GR-14 Chunker with stable ids and metadata

3 pts Delivery owner

rag_runtime/chunking.py: heading and paragraph aware, size cap, ids <doc>#<n>, keeps section and page. Chunk size is a design attribute.

ACCEPTANCE CRITERIA

- Same input gives the same ids
- Chunks never split mid-sentence in tests
- Section and page carried on every chunk

Depends on: GR-13

GR-15 Entity and relation extraction, live and offline

5 pts Delivery owner

Live: the generated EntityRelationExtractor signature via dspy. Offline: deterministic rule-based extractor (quoted defined terms, capitalised party names, 'clause n' references) behind the same interface.

ACCEPTANCE CRITERIA

- Offline extractor finds at least 80% of gold entities on the fixture
- Live extraction LM calls appear in the trace step
- Malformed model output is rejected and logged, never written

Depends on: GR-8, GR-10, GR-14

GR-16 Entity normalisation and alias merge

3 pts Delivery owner

Normalise keys (case, whitespace, punctuation, articles), merge aliases such as 'the Supplier' with the defined party, keep all surface names as aliases.

ACCEPTANCE CRITERIA

- Duplicate entity rate on the fixture below 5%
- Alias merge is deterministic and unit tested

Depends on: GR-15

GR-17 Graph writer for kg:<project> in FalkorDB

5 pts · Delivery owner

Write Document, Chunk, Entity nodes and HAS_CHUNK, MENTIONS, RELATED edges with parameterised, batched Cypher (UNWIND). Re-ingestion replaces the project's graph. Offline twin builds the same structure as an in-memory LPG.

ACCEPTANCE CRITERIA

- Counts in FalkorDB equal counts in the in-memory twin
- No string-built Cypher with user data
- Re-running ingestion is idempotent

Depends on: GR-16

GR-18 Indexes for entity linking

2 pts · Delivery owner

Create full-text index on Entity name and aliases; range indexes on Chunk.id and Entity.key.

ACCEPTANCE CRITERIA

- Index creation is part of ingestion and idempotent
- Full-text query returns gold entities for 10 sample terms

Depends on: GR-17

GR-19 cli ingest and owned kg graphs

3 pts · Delivery owner

cli ingest --project <id> runs the generated indexing program; launcher treats kg:* graphs as owned (left intact by cli up, listed by status); journal event ingest.finished; traced as the index step.

ACCEPTANCE CRITERIA

- cli ingest --project RS-11 builds kg:RS-11
- cli up does not delete kg graphs
- Trace folder NN-R1-index exists with graph counts

Depends on: GR-12, GR-17

GR-20 Ingestion report

2 pts · Delivery owner

design/ingest-report.json and .md: documents, chunks, entities, relations, extraction calls and tokens, duplicates merged, failures.

ACCEPTANCE CRITERIA

- Report written on every ingestion
- Token totals match the LM history

Depends on: GR-19

Week 3 · 16 to 20 November

8 stories · 24 points

Answer questions from the graph: link, expand, collect, cite. Start measuring.

GR-E4 · GRAPH RETRIEVER

GR-21 Entity linking from the question

3 pts · Delivery owner

Link question terms to entities through the full-text index (live) or token match over names and aliases (offline). Return ranked entity keys with scores.

ACCEPTANCE CRITERIA

- Linking recall at least 0.8 on the eval questions' gold entities
- Offline and live agree on the fixture for 10 sample questions

Depends on: GR-18

GR-22 Bounded neighbourhood expansion

3 pts · Delivery owner

Expand from linked entities up to maxHops over RELATED, capped at nodeCap nodes, with parameterised Cypher; offline traversal over the LPG.

ACCEPTANCE CRITERIA

- Never returns more than nodeCap nodes
- Hop depth honoured (unit test with a chain fixture)

Depends on: GR-21

GR-23 Chunk collection, ranking and provenance

3 pts · Delivery owner

Collect chunks that mention expanded entities or state their relations; score by linked-entity overlap and hop distance; return top k as chunks with source_id, text and score; put relation lines in state for the synthesizer.

ACCEPTANCE CRITERIA

- Every returned chunk has a source_id that exists in the graph
- Gold evidence chunk in top k for at least 70% of multi-hop questions

Depends on: GR-22

GR-24 Bind the graph retriever into the generated agent

2 pts · Delivery owner

rag_runtime.bind(module, project) sets IMPLEMENTATIONS['GraphRetriever']; the eval step uses rag_runtime when the project has a kg graph, otherwise the stub.

ACCEPTANCE CRITERIA

- RS-11 eval runs through the real retriever online and the twin offline
- RS-01 behaviour unchanged

Depends on: GR-23

GR-25 Parity test: in-memory retriever versus FalkorDB

3 pts · Delivery owner

On the fixture corpus, both backends return the same entities and chunk ids for the eval questions. Skips without FalkorDB.

ACCEPTANCE CRITERIA

- Parity on all eval questions when FalkorDB is up
- Skipped cleanly in CI

Depends on: GR-24

GR-26 Retrieval trace

2 pts · Delivery owner

step.json for eval records linked entities, hops, chunk ids per question; inspector renders them.

ACCEPTANCE CRITERIA

- Step Inspector shows linked entities and chunks for any question

Depends on: GR-24

GR-E5 · EVALUATION AND END TO END

GR-27 RS-11 eval set

3 pts · Architecture owner

30 questions: 10 single-hop, 10 multi-hop across documents, 5 definitions, 5 unanswerable, each with gold chunk ids and gold entities. Versioned with author and date.

ACCEPTANCE CRITERIA

- Every multi-hop question needs at least two documents
- Approved by the architecture owner

Depends on: GR-10

GR-28 Graph-aware metrics

5 pts · Delivery owner

Add citation recall@k, cross-document recall, entity-linking recall, abstention accuracy and groundedness, plus measured p95 latency and tokens per answer; each traced to an RS-11 requirement in the report.

ACCEPTANCE CRITERIA

- Metrics computed on the offline run with known expected values
- Report names the requirement behind each metric

Depends on: GR-27

Week 4 · 23 to 27 November

9 stories · 23 points

Prove it end to end, make it inspectable, and close with the review gate and a demo.

GR-E5 · EVALUATION AND END TO END

GR-29 Baseline comparison

2 pts · Delivery owner

Run the same eval set with the word-overlap baseline retriever and put both results side by side in the report.

ACCEPTANCE CRITERIA

- Report shows GraphRAG versus baseline per metric

Depends on: GR-28

GR-30 L2 oracle test test_rs_11.py

3 pts · Delivery owner

Assert the RS-11 oracle offline: selection, rejections, gate, budgets, numbers, graph content, programs, eval report. Anything this plan can't satisfy is a strict xfail naming its increment.

ACCEPTANCE CRITERIA

- Passes offline with no key and no FalkorDB
- models/ hash unchanged by the run

Depends on: GR-28

GR-31 L3 test with live model and FalkorDB

2 pts · Delivery owner

Mark slow. Ingest, run the eval set live, assert thresholds agreed in GR-27.

ACCEPTANCE CRITERIA

- Passes on the workbench machine with FalkorDB up and a configured model

Depends on: GR-29

GR-32 End-to-end acceptance run

2 pts · Architecture owner

cli run --project RS-11 --online on a clean checkout; review the run record, trace and decision log; record go or no-go as a Decision.

ACCEPTANCE CRITERIA

- All five definition-of-done conditions hold
- Decision recorded

Depends on: GR-31

GR-E6 · HARDENING, INSPECTION AND HANDOVER

GR-33 Workbench: knowledge-graph view for kg graphs

3 pts · Delivery owner

Knowledge Graph page can pick kg:<project> (read-only), search entities and show a neighbourhood; rendering lives in inspector.py.

ACCEPTANCE CRITERIA

- Entity search and neighbourhood view work for RS-11
- No write path from the app

Depends on: GR-19

GR-34 Generated GraphRAG notebook

3 pts · Delivery owner

Extend notebooks/build_i0.py (or a sibling builder) with a GraphRAG notebook: ingest, inspect graph, ask, trace, evaluate. Never edit the notebook by hand.

ACCEPTANCE CRITERIA

- test_notebooks.py executes it offline

Depends on: GR-24

GR-35 Agent container (stretch)

5 pts · Delivery owner · stretch

Generate a Dockerfile and compose file for the RS-11 agent with a REST endpoint on the shared FalkorDB network.

ACCEPTANCE CRITERIA

- Container answers an RS-11 question locally
- Byte-identical regeneration

Depends on: GR-32

GR-36 Documentation updates

2 pts · Architecture owner

Update ROADMAP.md, README.md, the team handbook and the system reference for GraphRAG; regenerate docs/ase-architecture.html.

ACCEPTANCE CRITERIA

- Docs match the code; no 36-checks drift introduced

Depends on: GR-32

GR-37 Review gate and demo

1 pts · Architecture owner

Run the roadmap's review checklist (models diff, selftest, orchestrator regen, tests, decision log) and demo the RS-11 agent.

ACCEPTANCE CRITERIA

- Checklist recorded in the project notes
- Demo held

Depends on: GR-36

Risks

RISK	EFFECT	MITIGATION
Extraction quality is poor on real documents	Missing entities, weak recall	Gold triples in the fixture corpus; entity recall metric from week 2; tune the extraction signature before week 3
Indexing cost on larger corpora	Slow or expensive ingestion	One call per chunk, batched; ingestion report shows tokens; corpus kept at 20 to 30 documents for this plan
In-memory Cypher subset can't run graph queries	Offline tests diverge from live behaviour	Offline retriever traverses the LPG in Python; a parity test compares it with FalkorDB on the fixture
Roadmap order (GraphRetriever planned for I5)	Design drift, double work in I5	DG-1 recorded; scope limited to single tenant; I5 extends rather than replaces
Codegen fixes change existing generated programs	Unexpected diffs in `generated/`	Regenerate in one PR, diff reviewed, notebooks re-run
Library change blurs the read-only rule	Review confusion	One labelled library commit by the architecture owner in week 1; no ASE run edits `models/`

Export to JIRA

The button copies every epic and story as CSV for JIRA's CSV importer (System → External system import → CSV). Map *Issue Id* and *Parent Id* to link stories to epics; *Story Points* to your story-point field; *Sprint* to sprint names you create first.